

Near Neighbour techniques in cryptanalysis

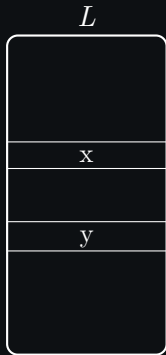
Elena Kirshanova

at Africacrypt 2026
Hammamet, Tunisia

Agenda

- I. The closest pair problem
- II. Algorithms for the closest pair problem
- III. What happens in practice
- IV. Open questions

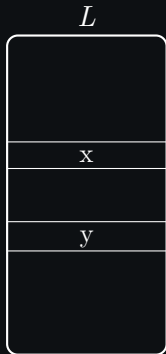
The closest pairs problem



Given a list L and $r > 0$, find all $x, y \in L$ such that

$$\text{dist}(x, y) < r.$$

The closest pairs problem

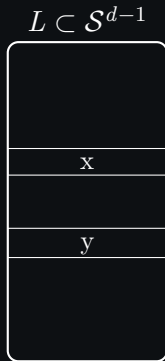


Given a list L and $r > 0$, find all $x, y \in L$ such that

$$\text{dist}(x, y) < r.$$

- Cases of interest:
 - ▶ $L \subset \mathcal{S}^{d-1}$ – a unit sphere, $r = \Theta(1)$, $\text{dist}(x, y) = \langle x, y \rangle$
 - ▶ $L \subset \{0, 1\}^d$, $r = \Theta(d)$, $\text{dist}()$ is Hamming
- Often $|L| = \exp(d)$ (dense setting)
- Common assumption: elements in L are uniformly distributed

Why interesting?

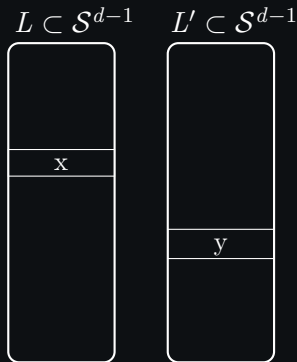


Main subroutine inside sieving algorithms for the **Shortest** Vector Problem [BGJ15], [BDGL16]

For $|L| = \left(\frac{4}{3}\right)^{d/2}$, we can heuristically solve this problem in $T = \left(\frac{3}{2}\right)^{d/2}$ time and $S = \left(\frac{4}{3}\right)^{d/2}$ space.

These complexities are used to setup concrete parameters for all lattice-based constructions.

Why interesting?



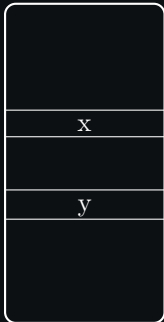
Main subroutine inside Randomized Slicer algorithms for the **Closest** Vector Problem [DLdW19, DLvW20]

For $|L| = \left(\frac{4}{3}\right)^{d/2}$, we can heuristically solve this problem in $T = \left(\frac{3}{2}\right)^{d/2}$ time and $S = \left(\frac{4}{3}\right)^{d/2}$ space.

All $o(d)$ factors in the exponents are omitted

Why interesting?

$$L \subset \{0,1\}^d$$

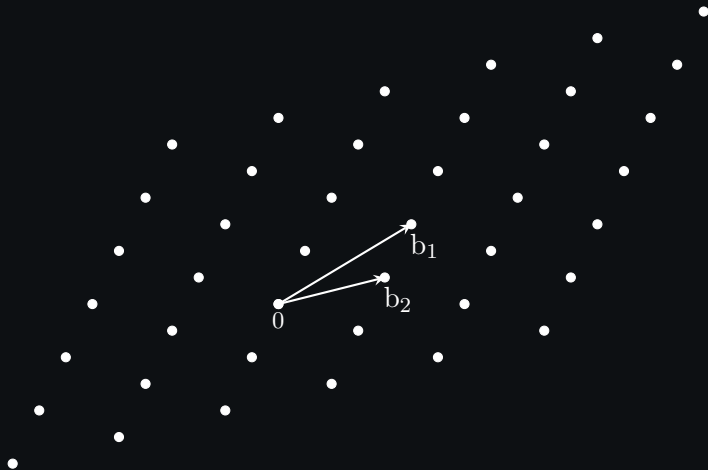


Main subroutine inside Information Set
Decoding algorithms, [MO15], [BM18]

as well as in sieving algorithms for decoding
[GJN23], [DEEK24]

Relevant to the dense error decoding setting:
 $\text{dist}(x, y) = \Theta(d)$

Let's come back to SVP



Let's come back to SVP



Given $\{b_1, \dots, b_d\}$ – a basis of $\mathcal{L}$
the **Shortest Vector Problem (SVP)**
asks to find non-zero v
of minimal length.

We do not know $\|v_{\text{shortest}}\|$ in general, but for any d -rank $\mathcal{L}$:

$$\|v_{\text{shortest}}\| \leq \sqrt{d} \cdot \det(\mathcal{L})^{1/d} \quad (\text{Minkowski's bound})$$

Algorithms for SVP

- Enumeration

- Sieving

Algorithms for SVP

- Enumeration

$$\log_2(\text{Time}) = ((1/2e) + o(1)) \cdot d \log d \quad \text{Memory} = \text{poly}(d)$$

- ✓ Lots of improvements for the $o(d \log d)$ -term
- ✓ Easy to parallelize
- ✓ Up until 2019 was the only practical SVP algorithm

- Sieving

Algorithms for SVP

- Enumeration

$$\log_2(\text{Time}) = ((1/2e) + o(1)) \cdot d \log d \quad \text{Memory} = \text{poly}(d)$$

- ✓ Lots of improvements for the $o(d \log d)$ -term
- ✓ Easy to parallelize
- ✓ Up until 2019 was the only practical SVP algorithm

- Sieving

$$\text{Provable: } \log_2(\text{Time}) = d + o(d) \quad \log_2(\text{Memory}) = d + o(d)$$

$$\text{Heuristic: } \log_2(\text{Time}) = (0.292 + o(1))d \quad \log_2(\text{Memory}) = (0.2075 + o(1))d$$

- ✓ Big $o(d)$ -factors
- ✓ Significant amount of work has been put in estimating and improving $o(d)$ -factors
- ✓ Parallelization is less straightforward (but possible)

Algorithms for SVP

- Enumeration

$$\log_2(\text{Time}) = ((1/2e) + o(1)) \cdot d \log d \quad \text{Memory} = \text{poly}(d)$$

- ✓ Lots of improvements for the $o(d \log d)$ -term
- ✓ Easy to parallelize
- ✓ Up until 2019 was the only practical SVP algorithm

- Sieving

Provable: $\log_2(\text{Time}) = d + o(d)$

$\log_2(\text{Memory}) = d + o(d)$

Heuristic: $\log_2(\text{Time}) = (0.292 + o(1))d$

$\log_2(\text{Memory}) = (0.2075 + o(1))d$

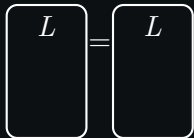
This talk



- ✓ Big $o(d)$ -factors
- ✓ Significant amount of work has been put in estimating and improving $o(d)$ -factors
- ✓ Parallelization is less straightforward (but possible)

Nguyen-Vidick sieve [NV08]

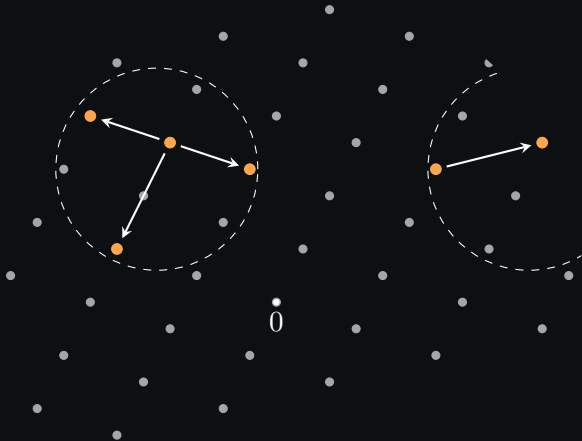
Idea: saturate space with enough lattice vectors so that their sums give short(er) vectors



Nguyen-Vidick sieve [NV08]

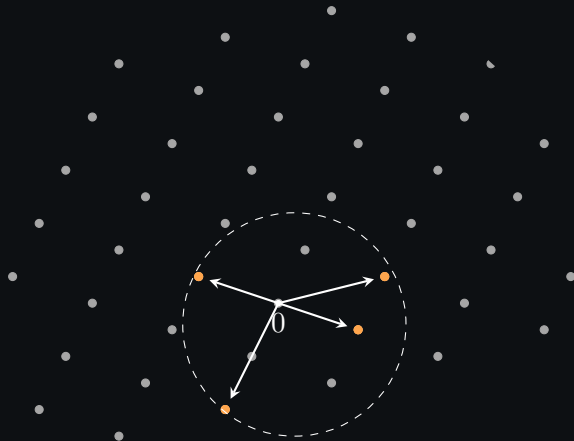
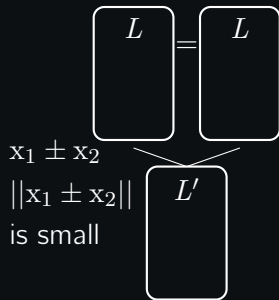
Idea: saturate space with enough lattice vectors so that their sums give short(er) vectors

$$L = L$$



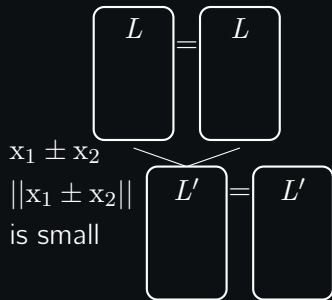
Nguyen-Vidick sieve [NV08]

Idea: saturate space with enough lattice vectors so that their sums give short(er) vectors



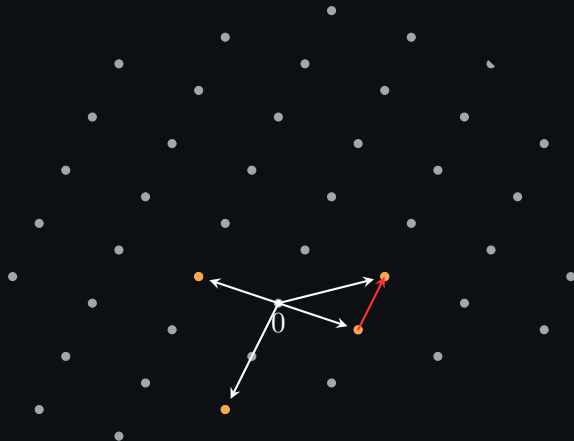
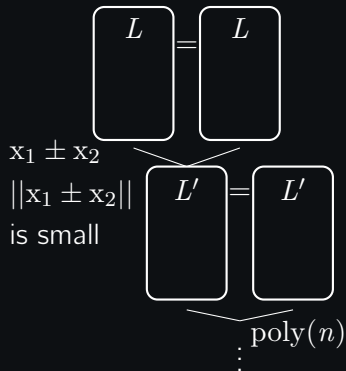
Nguyen-Vidick sieve [NV08]

Idea: saturate space with enough lattice vectors so that their sums give short(er) vectors



Nguyen-Vidick sieve [NV08]

Idea: saturate space with enough lattice vectors so that their sums give short(er) vectors



How large $|L|$ should be?

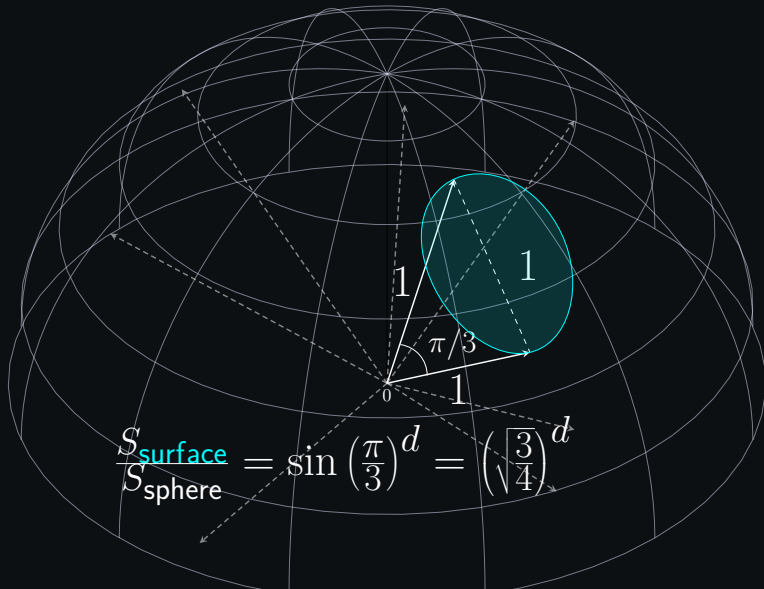
Assumption: vectors (normalized) in $|L|$ are uniform iid
on S^{d-1} .

This assumption is the major difference between heuristic and provable algorithms

How large $|L|$ should be?



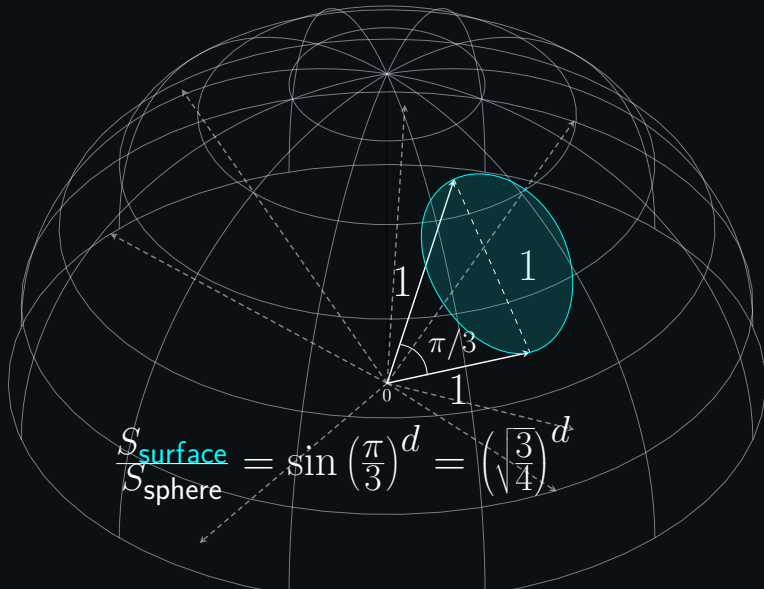
How large $|L|$ should be?



$$\frac{S_{\text{surface}}}{S_{\text{sphere}}} = \sin\left(\frac{\pi}{3}\right)^d = \left(\frac{\sqrt{3}}{2}\right)^d$$

How large $|L|$ should be?

$\left(\sqrt{\frac{4}{3}}\right)^d$ vectors cover S^{d-1}



$$\frac{S_{\text{surface}}}{S_{\text{sphere}}} = \sin\left(\frac{\pi}{3}\right)^d = \left(\frac{\sqrt{3}}{2}\right)^d$$

How to solve the closest pairs problem on $\mathcal{S}^{d-1}$?

Use locality-sensitive hashing (LSH).

LSH is built upon a family of hash functions h such that

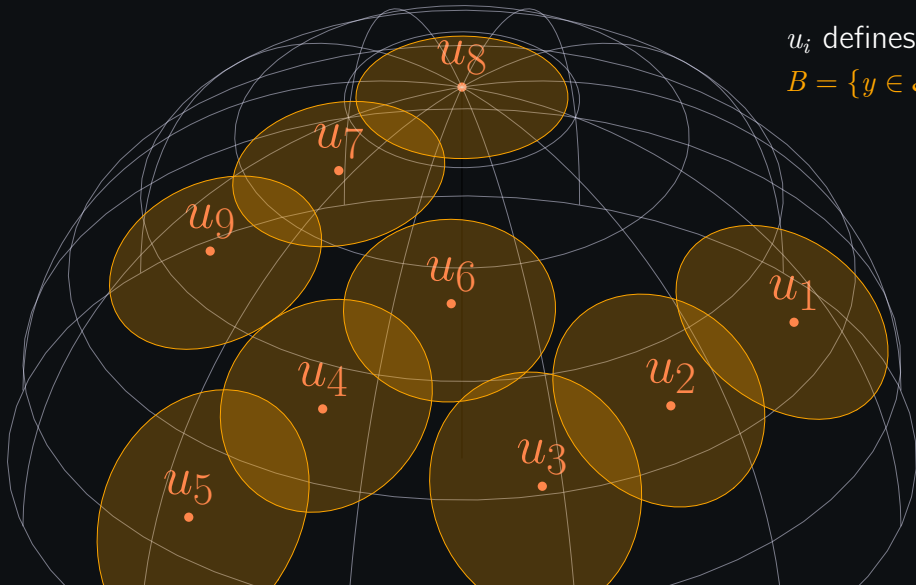
$$\Pr_{\substack{\mathbf{x}, \mathbf{y} \sim \mathcal{S}^{d-1} \\ \text{dist}(\mathbf{x}, \mathbf{y}) < r}} [h(\mathbf{x}) = h(\mathbf{y})] \gg \Pr_{\mathbf{x}, \mathbf{y} \sim \mathcal{S}^{d-1}} [h(\mathbf{x}) = h(\mathbf{y})]$$

Build hash tables (buckets) with a “nice” hash function, run search for close pairs within each bucket.

LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

u_i defines a 'bucket' B

$$B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| > \alpha\}$$

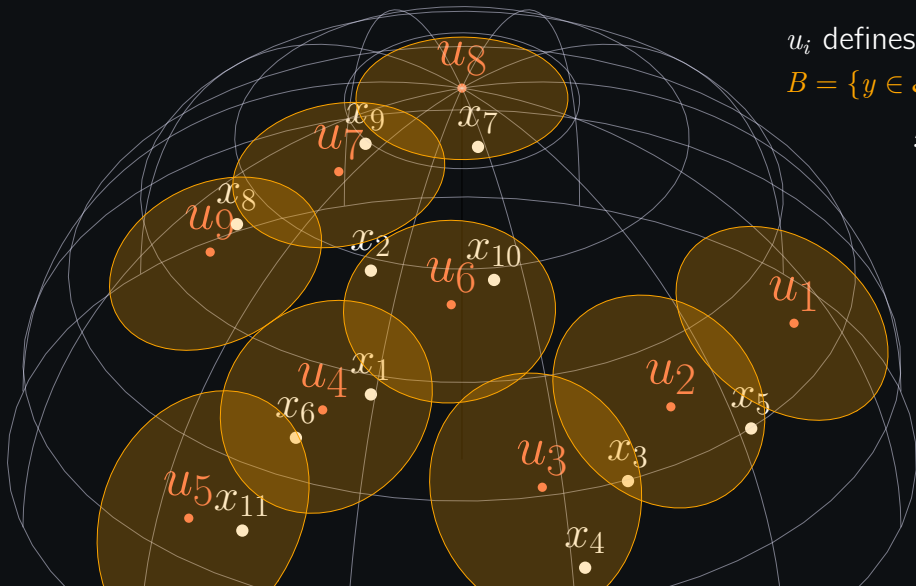


LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

u_i defines a 'bucket' B

$$B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| \geq \alpha\}$$

$$x_i \in L$$



LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

For all $x \in L$:

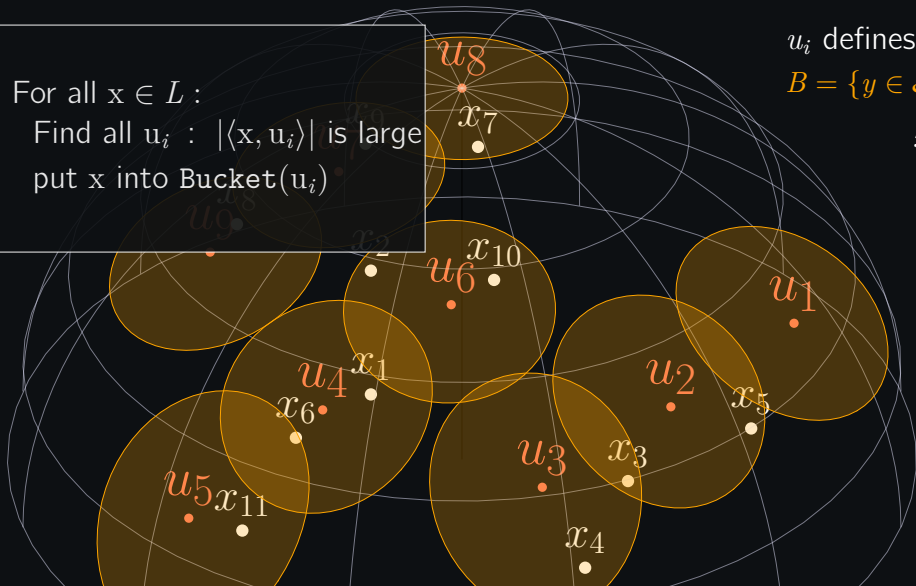
Find all u_i : $|\langle x, u_i \rangle|$ is large

put x into $\text{Bucket}(u_i)$

u_i defines a 'bucket' B

$$B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| \geq \alpha\}$$

$x_i \in L$



LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

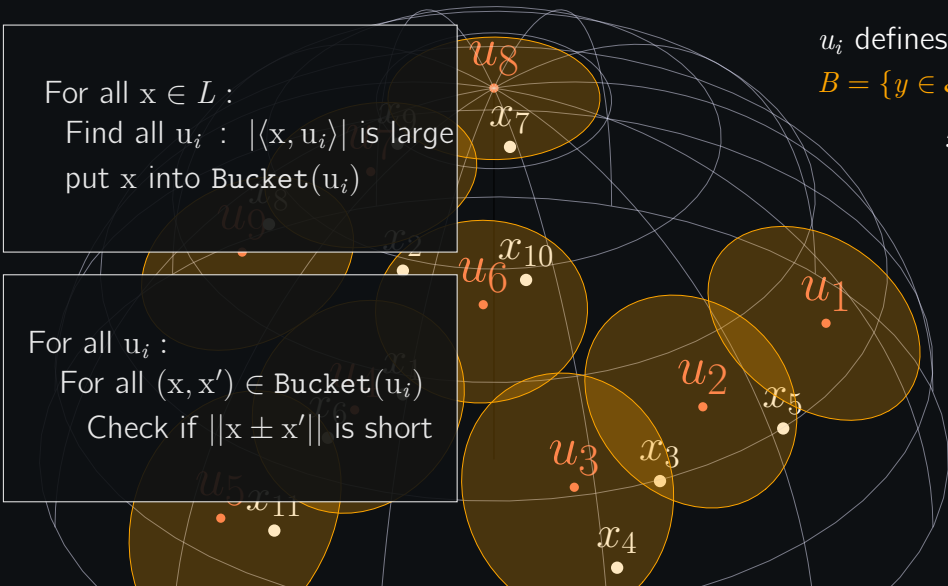
For all $x \in L$:
Find all u_i : $|\langle x, u_i \rangle|$ is large
put x into $\text{Bucket}(u_i)$

For all u_i :
For all $(x, x') \in \text{Bucket}(u_i)$
Check if $\|x \pm x'\|$ is short

u_i defines a 'bucket' B

$$B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| \geq \alpha\}$$

$x_i \in L$



LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

For all $x \in L$:

Find all u_i : $|\langle x, u_i \rangle|$ is large
put x into $\text{Bucket}(u_i)$

For all u_i :

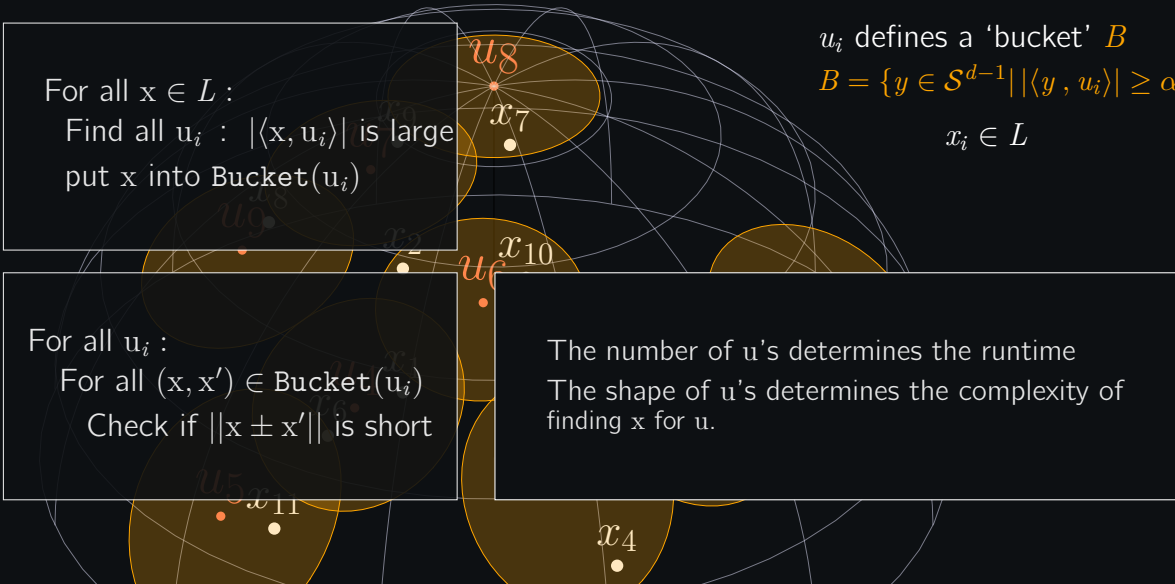
For all $(x, x') \in \text{Bucket}(u_i)$
Check if $\|x \pm x'\|$ is short

u_i defines a 'bucket' B

$$B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| \geq \alpha\}$$

$x_i \in L$

The number of u 's determines the runtime
The shape of u 's determines the complexity of finding x for u .



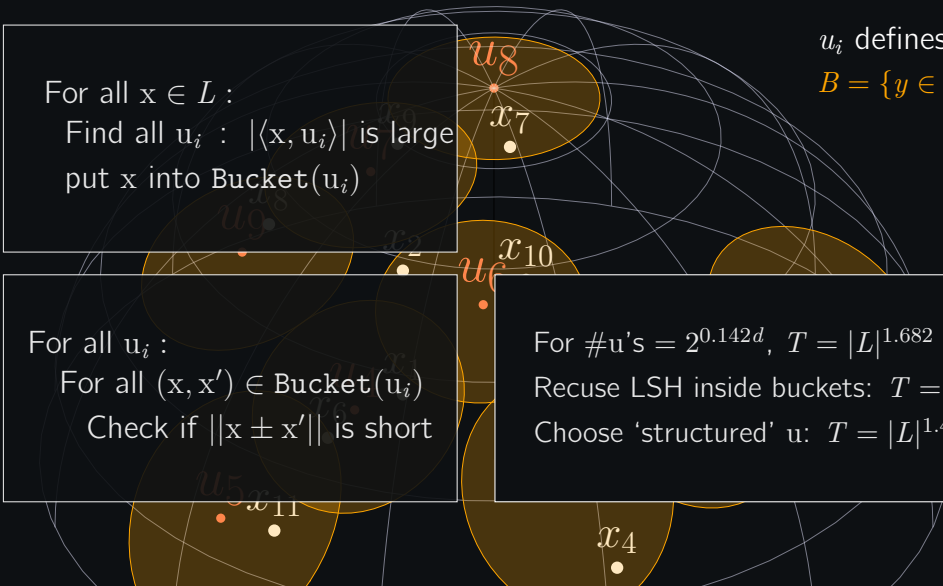
LSH on $\mathcal{S}^{d-1}$ [BGJ15, BDGL16]

For all $x \in L$:
Find all u_i : $|\langle x, u_i \rangle|$ is large
put x into $\text{Bucket}(u_i)$

u_i defines a 'bucket' B
 $B = \{y \in \mathcal{S}^{d-1} \mid |\langle y, u_i \rangle| \geq \alpha\}$
 $x_i \in L$

For all u_i :
For all $(x, x') \in \text{Bucket}(u_i)$
Check if $\|x \pm x'\|$ is short

For $\#u$'s = $2^{0.142d}$, $T = |L|^{1.682}$
Recuse LSH inside buckets: $T = |L|^{1.5}$, [BGJ15]
Choose 'structured' u : $T = |L|^{1.407}$, [BDGL16].



Theoretical lower bounds [KL'21]

Instantiating LSH with Spherical caps, i.e.,

$$B_{\mathbf{x}}(\alpha) := \{y \in \mathcal{S}^{d-1} : \langle \mathbf{x}, y \rangle \geq \alpha\},$$

is optimal in the **Euclidean metric** and almost optimal in the **Hamming metric**.

Here **optimal** means that choosing hash regions different from spherical caps will not **asymptotically** improve the performance of LSH.

Theoretical lower bounds [KL'21]

Instantiating LSH with Spherical caps, i.e.,

$$B_{\mathbf{x}}(\alpha) := \{y \in \mathcal{S}^{d-1} : \langle \mathbf{x}, y \rangle \geq \alpha\},$$

is optimal in the **Euclidean metric** and almost optimal in the **Hamming metric**.

Here **optimal** means that choosing hash regions different from spherical caps will not **asymptotically** improve the performance of LSH.

Consequences:

- Another hashing strategy will not improve the performance of lattice sieving
- Improving only the closest pair subroutine in ISD will not result in a noticeable gain
- Asymptotically fastest algorithm will choose $\mathbf{x}$'s from a fast-decodable spherical code (may not exist for arbitrary dimensions).

Interpretation of the lower bounds

- The result does not imply a lower bound on all possible **sieving algorithms**. Another use of the closest pairs problem or a completely different technique is possible.
- It implies that we have an optimal near neighbor subroutine within sieving algorithms.
- Lower bounds apply to lists of exponential (in d) size. Developing interesting sieving with subexponential size lists is an open problem.

SVP in practice

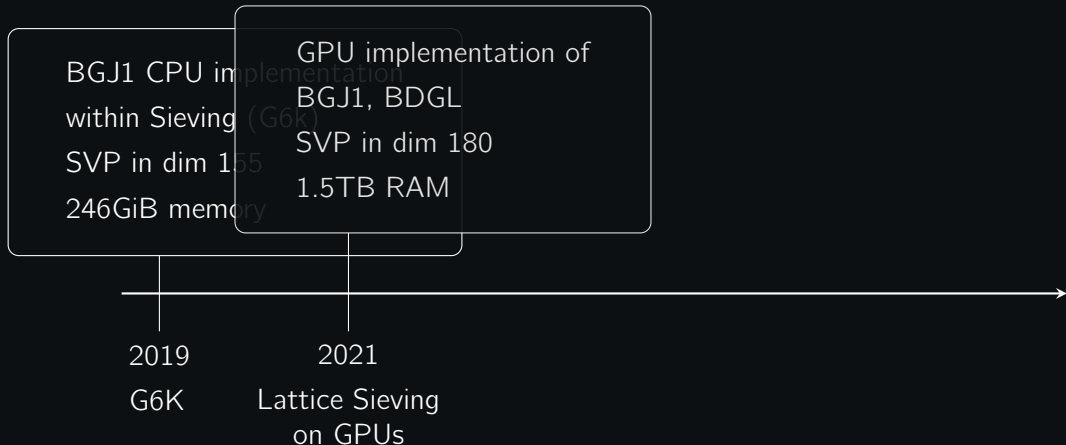
BGJ1 CPU implementation
within Sieving (G6k)
SVP in dim 155
246GiB memory

2019

G6K

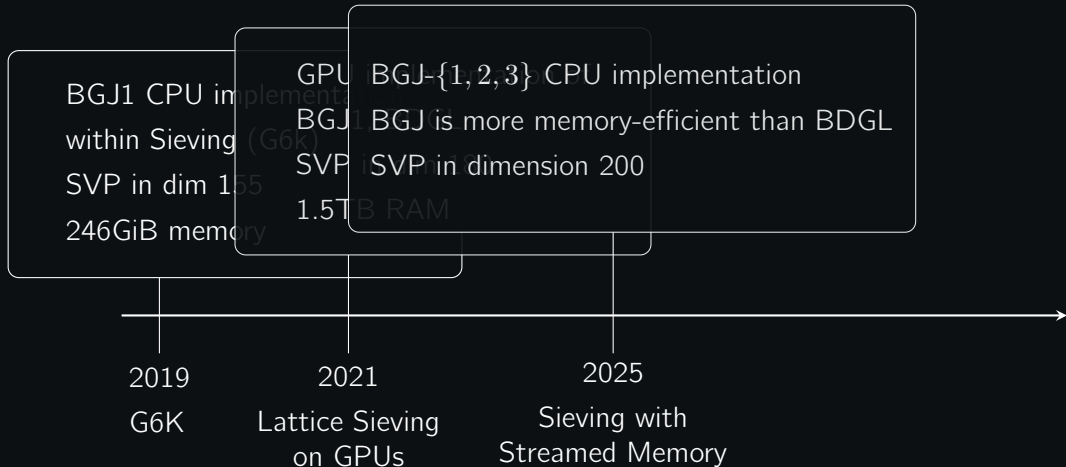
Checkout Darmstadt's SVP challenge <https://www.latticechallenge.org/svp-challenge/halloffame.php>

SVP in practice



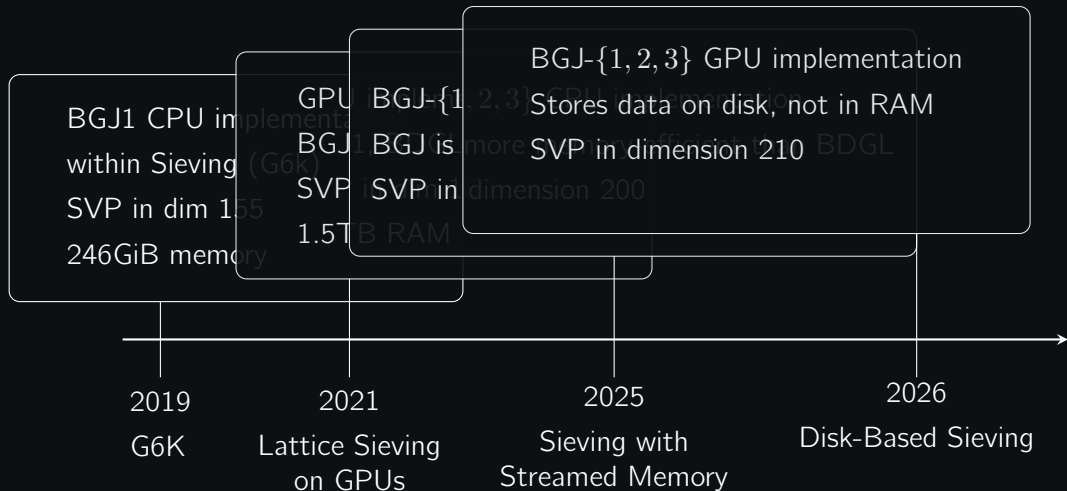
Checkout Darmstadt's SVP challenge <https://www.latticechallenge.org/svp-challenge/halloffame.php>

SVP in practice



Checkout Darmstadt's SVP challenge <https://www.latticechallenge.org/svp-challenge/halloffame.php>

SVP in practice



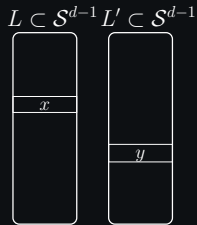
Checkout Darmstadt's SVP challenge <https://www.latticechallenge.org/svp-challenge/halloffame.php>

CVP in practice

BDGL CPU implementation within G6K
NOT aiming for records ($d \leq 100$)
A subroutine in hybrid algs. against LWE

2025

Randomized Slicer for CVP



Main challenges in practice

- Huge memory requirement
- Communication cost
- Large small-order terms affect performance

Open questions

- Memory efficient, time optimal Near Neighbour technique
- Nearest Neighbor for the planted close pair.
- Closest pairs problem in other norms like ℓ_∞ .
- Lower bounds apply to hash-based approach to find close pairs. Other approaches?
- Should large memory requirement be taken into account in security estimates?¹
- Improve implementation for close pair from different lists L, L' (CVP case)

¹See S. Jaques "Memory adds no cost to lattice sieving for computers in 3 or more spatial dimensions"

References

- [G6K] M.R. Albrecht, L. Ducas, G. Herold, E. Kirshanova, E.W. Postlethwaite, and M. Stevens, 2019. The general sieve kernel and new records in lattice reduction.
- [BDJ15] A. Becker, N. Gama, A.Joux. Speeding-up lattice sieving without increasing the memory, using sub-quadratic nearest neighbor search
- [BDGL16] A. Becker, L. Ducas, N. Gama, T. Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving.
- [DEEK24] L. Ducas, A. Esser, S. Etinski, E. Kirshanova. Asymptotics and Improvements of Sieving for Codes.
- [NV08] P. Nguyen, T. Vidick. Sieve algorithms for the shortest vector problem are practical
- [MO15] A. May, I.Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes.